ESGI – Master 1 Architecture Logicielle

Module : CI/CD avec Jenkins

Rapport technique

Chaîne CI/CD complète sous Jenkins

Application : Job Application Tracker

Équipe

Basma MORABIT
Bastien LIEVRE
Siham AIDROUS
Mariam FATHALLAH

Dépôt : `https://github.com/mariamfathallah/job-application-tracker`

1er novembre 2026

Table des matières

Introduction **2**

1 Architecture **4**
1.1 Vue d'ensemble 4
1.2 Rôle de chaque composant 4
1.3 Démarrage en une commande 4

2 Choix techniques et justifications **6**
2.1 Partie commune 6
2.2 Qualité et retour rapide 6
2.3 Releases automatisées 7
2.4 Blue/green 8
2.5 Environnements as Code 9
2.6 Observabilité et DORA 9

3 Pipeline détaillé **11**
3.1 Les étapes du Jenkinsfile 11
3.2 Déclenchement 12
3.3 Shared Library 13
3.4 Gestion des secrets 13

4 Environnements et déploiement **15**
4.1 Staging et production 15
4.2 Environnements as Code (OpenTofu et Ansible) 15
4.3 Déploiement blue/green 15
4.4 Idempotence et même artefact 16

5 Qualité **18**
5.1 Tests et couverture 18
5.2 SonarQube et Quality Gate 18
5.3 Durée du pipeline 18

6 Métriques DORA **20**
6.1 Chaîne de mesure 20
6.2 Définitions et calculs 20
6.3 Résultats 21
6.4 Analyse 21
6.5 Limites 22

7 Difficultés rencontrées et solutions **23**

8 Répartition du travail **24**

Conclusion **25**

Introduction

Contexte et objectif

Livrer souvent sans casser la production suppose d'automatiser tout ce qui se passe entre un commit et sa mise en ligne. C'est l'objet de ce projet, réalisé dans le cadre du module *CI/CD avec Jenkins* : concevoir une chaîne d'intégration et de livraison continues complète sous Jenkins. Chaque commit y est construit, testé, analysé, empaqueté en un artefact versionné, puis livré en staging et, après validation humaine, en production.

L'enjeu n'est pas l'application elle-même, mais la chaîne qui l'entoure : elle doit être reproductible, rapide, fiable, et entièrement décrite dans le dépôt Git (*Pipeline as Code*, configuration de Jenkins par code, environnements décrits par code).

L'application

La chaîne est construite autour d'une application existante, *Job Application Tracker*, qui permet de suivre ses candidatures tout au long d'un processus de recrutement. Elle se compose de trois parties :

— une API REST en Java avec Spring Boot, sécurisée par JWT, organisée en couches (contrôleurs, services, dépôts) ;

— une interface web en React ;

— une base de données PostgreSQL, dont le schéma est géré par Liquibase.

Elle offre un terrain réaliste pour une chaîne CI/CD : deux langages à tester, une base de données pour les tests d'intégration, et une sonde de santé (Spring Boot Actuator) sur laquelle s'appuient les déploiements.

Périmètre réalisé

L'équipe, composée de quatre membres, a réalisé la partie commune exigée et cinq modules, dont les deux de niveau 3 :

— **partie commune** : outillage démarré en une commande (Jenkins, SonarQube, Nexus), builds exécutés sur un agent, job Multibranch déclenché par webhook, statut du build sur les pull requests, tests et Quality Gate bloquante, artefact versionné publié dans Nexus, déploiement automatique en staging puis en production après validation, Shared Library versionnée par tag, secrets dans les credentials Jenkins ;

— **Qualité et retour rapide** (niveau 1) ;

— **Releases automatisées** (niveau 1) ;

— **Déploiement blue/green** (niveau 2) ;

— **Environnements as Code** avec OpenTofu et Ansible (niveau 3) ;

— **Observabilité et métriques DORA** avec Prometheus et Grafana (niveau 3).

Plan du rapport

Le chapitre 1 présente l'architecture de la chaîne et le rôle de chaque composant. Le chapitre 2 justifie les choix techniques de chaque module et les alternatives écartées. Les chapitres 3 et 4 détaillent le pipeline puis les environnements et le déploiement. Le chapitre 5 porte sur la qualité,

et le chapitre 6 sur les métriques DORA mesurées pendant le projet. Les deux derniers chapitres reviennent sur les difficultés rencontrées et sur la répartition du travail.

1 Architecture

1.1 Vue d'ensemble

La chaîne complète tient dans le dépôt Git : un push déclenche le pipeline, qui construit, vérifie, publie puis déploie l'application dans deux environnements, pendant que Prometheus et Grafana mesurent l'ensemble.

Listing 1.1 – Architecture de la chaîne CI/CD

```
git push / PR --> GitHub --> webhook (smee) --> Jenkins controller (0 executor)
                                                          |
                                                          v
                  agent-1 : JDK 21, Node 22, Docker, OpenTofu, Ansible
  commit lint (PR) -> lint + tests -> SonarQube + Quality Gate || tests d'integration
  -> release (tag vX.Y.Z) -> jar X.Y.Z-<hash> -> Nexus
  -> plan OpenTofu -> staging (auto) -> validation manuelle -> production

  staging / production : reseau Docker + PostgreSQL + proxy nginx + blue/green
  Prometheus <-- Jenkins, staging, production, exporter DORA --> Grafana
```

1.2 Rôle de chaque composant

Composant	Rôle
Jenkins (controller)	orchestre les builds, ne build jamais lui-même (0 executor)
Agent `agent-1`	exécute tous les builds (JDK, Node, Docker, OpenTofu, Ansible)
SonarQube	analyse le code, Quality Gate bloquante
Nexus	stocke les jar versionnés
smee	relaie les webhooks GitHub vers notre Jenkins local
Staging / Production	environnements où tourne l'application (ports 8082 et 8083)
Prometheus / Grafana	collecte et affichage des métriques

Table 1.1 – Composants de la plateforme

1.3 Démarrage en une commande

Toute la plateforme est décrite dans un seul fichier, `ci/docker-compose.yml`, et démarre avec `docker compose up -d -build`. Rien n'est configuré à la main dans les interfaces : un membre de l'équipe, ou un correcteur, obtient la même plateforme à partir du dépôt et d'un fichier `ci/.env`.

— **Jenkins** est construit avec sa liste de plugins (`ci/jenkins/plugins.txt`) et configuré par **JCasC** (`ci/jenkins/casc.yaml`) : utilisateur administrateur, agent, credentials, Shared Library et job Multibranch sont créés au démarrage. Les secrets viennent des variables de `ci/.env`, jamais du dépôt.

— **L'agent** est une image dédiée (JDK 21, Node 22, client Docker, OpenTofu 1.10.6, Ansible), qui se connecte seule au controller. Il utilise le Docker de l'hôte (*docker-out-of-docker*) : son workspace est monté au même chemin (`/opt/jenkins-agent`) côté hôte et côté agent, pour que les conteneurs lancés par les builds voient les mêmes fichiers.

— **Volumes** : les caches Maven et npm accélèrent les builds, et l'état OpenTofu (`tf_state`) survit aux redémarrages.

— **Initialisation** : deux scripts idempotents complètent ce que Docker ne peut pas faire seul, la création du dépôt d'artefacts dans Nexus (`bootstrap-nexus.sh`) et le webhook SonarQube vers Jenkins (`bootstrap-sonar.sh`).

Tout étant décrit par code, chaque changement de la plateforme passe par une pull request relue, et la procédure complète de remise en route est donnée dans le `README.md` du dépôt.

2 Choix techniques et justifications

2.1 Partie commune

Le job Multibranch utilise la source **GitHub** (*GitHub Branch Source*), déclarée dans `ci/jenkins/casc.yaml`. Jenkins découvre ainsi les branches et les pull requests (builds `PR-<n>`) et publie le résultat de chaque build sur la PR. Jenkins tournant en local, il n'est pas joignable depuis Internet : le webhook GitHub passe par un relais **smee.io**, exécuté dans le conteneur `ci-smee` de la stack.

Choix	**Alternative écartée**	**Raison**
Source GitHub	Source Git simple	la source Git ne voit pas les pull requests et ne publie pas de statut
smee.io dans un conteneur	ngrok, Tailscale	vu en TP, aucun compte ni ouverture réseau ; le relais démarre avec le reste de la stack, sans terminal à laisser ouvert
Un Jenkins de référence (token en écriture)	Chaque membre publie ses statuts	quatre Jenkins écrivant le même check rendraient le statut ambigu ; les autres membres utilisent un token en lecture seule

Table 2.1 – Choix de l'intégration GitHub

Shared Library. Les étapes communes (publication dans Nexus, déploiement, notification) sont regroupées dans un dépôt séparé, `jenkins-shared-library`, plutôt que copiées dans le Jenkinsfile. Une correction se fait ainsi à un seul endroit. La bibliothèque est déclarée par code dans `casc.yaml` et chargée par `@Library('jenkins-shared-library@v1.2.0')`. Une bibliothèque globale s'exécute hors sandbox, avec une confiance totale : on l'épingle donc sur un tag, pour que modifier sa branche principale ne change aucun pipeline.

2.2 Qualité et retour rapide

Besoin	**Choix**	**Alternative écartée**	**Raison**
Mesurer la couverture	JaCoCo	Cobertura	intégré à Maven, lu par Jenkins et SonarQube
Imposer un minimum	règle `check` de JaCoCo	contrôle manuel	le build échoue automatiquement sous le seuil
Publier les rapports	plugins JUnit et Coverage	lecture des fichiers à la main	résultats visibles sur la page du build
Notifier	webhook Discord	e-mail, Slack	une simple URL, aucun plugin
Aller vite	parallélisme, caches Maven et npm	exécution en séquence	caches montés en volumes sur l'agent

Table 2.2 – Choix techniques du module Qualité

2.3 Releases automatisées

Les messages de commit suivent la convention *Conventional Commits*, vérifiée par **commitlint** dès la première étape du pipeline, sur les pull requests : un message invalide fait échouer le build immédiatement. Une fois tous les contrôles passés, **semantic-release** analyse les commits depuis le dernier tag et en déduit la version : `fix` et `perf` donnent un correctif, `feat` une version mineure, `BREAKING CHANGE` une version majeure. Il génère le changelog, crée le tag `vX.Y.Z` et publie une GitHub Release avec les notes de version. L'artefact publié dans Nexus est nommé `X.Y.Z-<hash>` : version issue du tag et hash du commit.

L'étape *Release* ne s'exécute que sur `main`, après les tests, la Quality Gate et les tests d'intégration : seul du code validé reçoit un numéro de version. Elle est de plus réservée au Jenkins de référence (`RELEASES_ENABLED=true`), pour qu'un seul Jenkins crée les tags.

Choix	**Alternative écartée**	**Raison**
semantic-release	release-please	release-please fonctionne par « PR de release » et est pensé pour GitHub Actions ; semantic-release s'exécute en une commande dans le pipeline
Vérification en CI	Hook Git local (husky)	un hook local peut être contourné ; la CI s'applique à tous
Pas de commit du `CHANGELOG.md`	Plugin `@semantic-release/git`	le commit relancerait un build et serait bloqué par la protection de `main` ; le changelog est archivé dans Jenkins et figure dans la GitHub Release
Commentaires automatiques désactivés	Comportement par défaut	la première release couvre tout l'historique : le plugin aurait commenté toutes les anciennes PR du dépôt

Table 2.3 – Choix du module Releases automatisées

Pendant le projet, neuf versions ont été calculées et publiées automatiquement :

Version	**PR**	**Type du commit**	**Effet**
v1.0.0	#12	`docs`	première release, calculée sur tout l'historique
v1.1.0	#13	`feat`	version mineure
v1.1.1	#14	`perf`	correctif
v1.1.2	#16	`fix`	correctif
v1.2.0	#15	`feat`	version mineure
v1.2.1	#17	`perf`	correctif
v1.3.0	#18	`feat`	version mineure
v1.3.1	#19	`fix`	correctif
v1.4.0	#23	`feat`	version mineure

Table 2.4 – Versions publiées par semantic-release

Les commits `ci`, `docs` et `chore` suivants (#21, #22, #24, #25) n'ont pas produit de version : ils ne modifient pas le livrable.

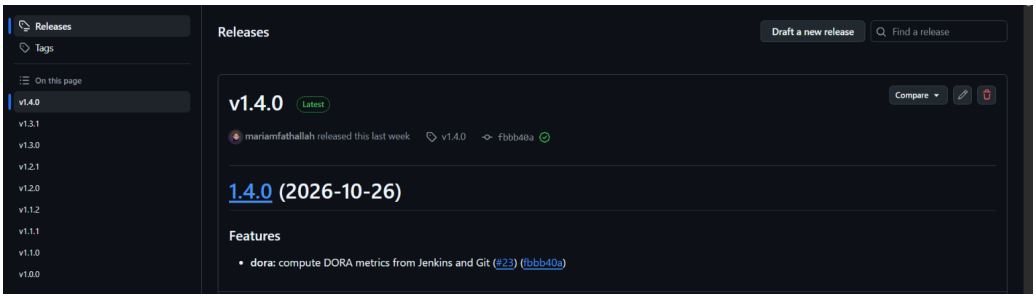


FIGURE 2.1 – Versions publiées automatiquement par semantic-release

2.4 Blue/green

Chaque environnement comporte un proxy **nginx**, seul port publié, et deux conteneurs applicatifs `blue` et `green` sans port publié. La nouvelle version est toujours déployée sur la couleur inactive, vérifiée, puis le proxy bascule. Le déroulé complet et le retour arrière sont décrits en section 4.3.

Choix	**Alternative écartée**	**Raison**
nginx	Traefik	configuration simple et lisible ; bascule par rechargement, sans coupure
Proxy géré par Ansible	Proxy décrit dans OpenTofu	nginx refuse de démarrer si sa cible n'existe pas encore : l'ordre « application puis proxy » se gère naturellement dans un playbook ; OpenTofu reste consacré à l'infrastructure durable
Blue/green en staging et en production	Production seule	staging doit être une copie fidèle de la production ; la stratégie est testée à chaque déploiement
Couleur active mémorisée dans le proxy	Fichier sur l'agent	l'agent n'est pas persistant ; le proxy est la source de vérité du trafic

TABLE 2.5 – Choix du module Blue/green

2.5 Environnements as Code

Choix	Alternative écartée	Raison
OpenTofu	Terraform	même langage et mêmes providers, mais licence libre (MPL) ; Terraform est passé sous licence BSL
Provider Docker (`kreuzwerker/docker`)	Machines virtuelles dans un cloud	aucun compte ni coût, environnements recréables sur chaque poste ; passer au cloud reviendrait à changer de provider, la structure restant la même
OpenTofu pour l'infrastructure, Ansible pour l'application	Un seul outil pour tout	l'infrastructure (réseau, base, secrets) change rarement ; l'application change à chaque livraison, ce qui relève de la configuration
Plan enregistré puis appliqué tel quel	`tofu apply` direct	ce qui a été relu est exactement ce qui est appliqué ; si l'infrastructure a changé entre-temps, OpenTofu refuse le plan périmé
État dans un volume Docker de l'agent	État dans Git	l'état contient les mots de passe générés : il ne doit jamais être versionné
Secrets générés (`random_password`)	Secrets écrits dans des fichiers	aucun secret à créer ni à partager ; chaque environnement a les siens

Table 2.6 – Choix du module Environnements as Code

2.6 Observabilité et DORA

Le principe directeur : la mesure doit être automatique, reproductible et lancée avec le reste de l'outillage par un seul `docker compose up -d`.

Besoin	Choix	Alternative écartée	Raison
Collecter et stocker les métriques	Prometheus, rétention 180 jours	Envoi depuis le pipeline (Pushgateway)	Prometheus va chercher les métriques lui-même : un build qui plante n'empêche pas la mesure
Afficher les tableaux de bord	Grafana configuré par fichiers	Tableaux de bord créés à la main	Versionnés dans Git, identiques sur chaque poste et relus en PR
Métriques de Jenkins	Plugin `prometheus`	Aucune	Standard, une ligne dans `plugins.txt`
Métriques de l'application	Micrometer (`/actuator/prometheus`)	Agent JMX	Déjà intégré à Spring Boot via Actuator
Calcul des métriques DORA	Exporter maison en Python	Plugins DORA, outils externes (Four Keys…)	Aucun plugin ne gère nos règles (validation manuelle, retour arrière blue/green, deux branches) ; les outils externes demandent un service hébergé

Table 2.7 – Choix techniques du module Observabilité et DORA

L'exporter n'utilise que la bibliothèque standard de Python, sans dépendance à installer. Il est couvert par 14 tests, dont un test de bout en bout contre un faux Jenkins.

3 Pipeline détaillé

3.1 Les étapes du Jenkinsfile

Étape	Quand	Rôle
Commit lint	pull request	messages au format Conventional Commits
Lint & tests	toujours	backend et frontend en parallèle
Analyse & intégration	toujours	SonarQube et tests d'intégration en parallèle
Release	`main`	version, tag, changelog
Package & publish	toujours	artefact publié dans Nexus
Infra plan	toujours	plan OpenTofu
Publish plan on PR	pull request	plan publié en commentaire
Deploy staging	`main`	déploiement automatique
Deploy production	`main`	validation manuelle, puis déploiement

TABLE 3.1 – Étapes du pipeline

L'étape de mise en production montre comment le pipeline combine validation humaine, infrastructure et déploiement :

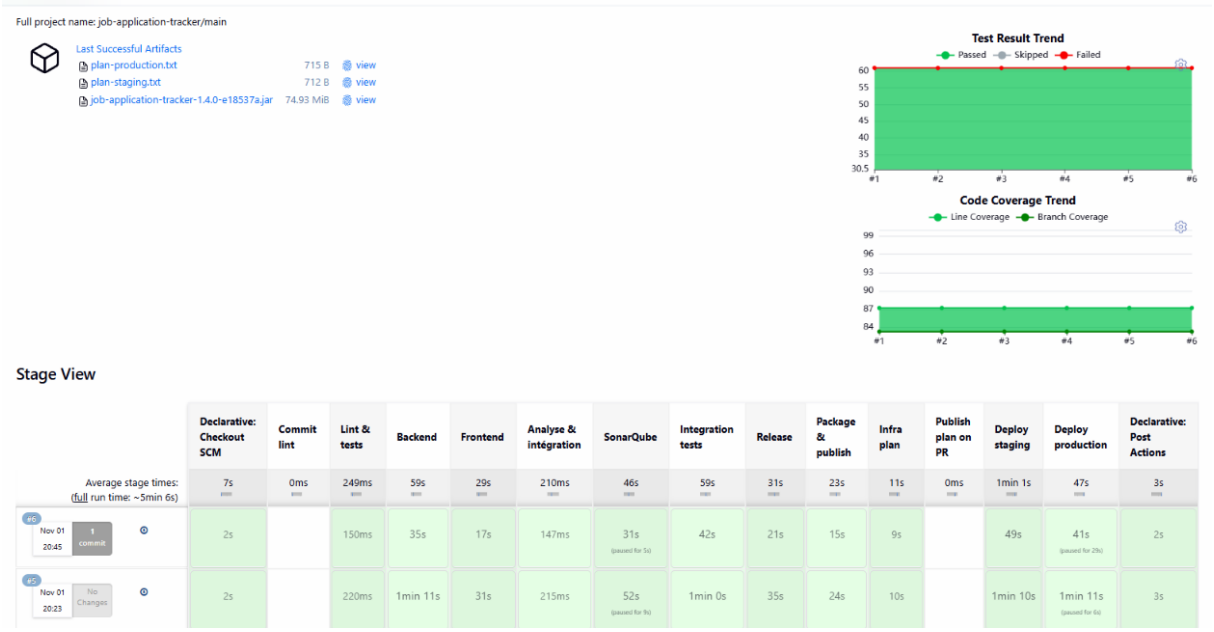


FIGURE 3.1 – Builds de `main` : toutes les étapes, jusqu'à la production

Listing 3.1 – Étape *Deploy production* du Jenkinsfile

```
stage('Deploy production') {
  when { expression { env.BRANCH_NAME == env.DEPLOY_BRANCH } }
  steps {
    timeout(time: 30, unit: 'MINUTES') {
      input message: "Déployer ${env.VERSION} en production ? (plan d'infra : artefact
     plan-production.txt)", submitter: 'admin'
    }
```

```
    sh 'sh ci/scripts/infra.sh apply production'   // applique exactement le plan
    validé
    deployTo('production')
  }
}
```

— `when` : seule la branche de déploiement (`main`) va jusqu'en production ;

— `input` avec `submitter` : seul l'utilisateur `admin` peut valider, en voyant la version et le plan d'infrastructure ; sans réponse sous 30 minutes, le build s'arrête sans déployer ;

— `infra.sh apply` applique le plan calculé plus tôt dans le même build, celui qui a été relu ;

— `deployTo` vient de la Shared Library et lance le déploiement blue/green.

3.2 Déclenchement

Un push sur GitHub déclenche un webhook, relayé par smee jusqu'à Jenkins (`/github-webhook/`) : le build démarre en environ deux secondes. Un scan du dépôt toutes les heures reste en place en secours, si un événement est perdu.

Sur une pull request, Jenkins construit le résultat de la fusion avec `main` et publie le statut du build sur la PR. La branche `main` est protégée :

— toute modification passe par une pull request ;

— une approbation d'un autre membre est obligatoire ;

— le check Jenkins doit être vert ;

— la fusion se fait uniquement en *squash*, ce qui garde un commit par PR et un historique propre pour semantic-release.

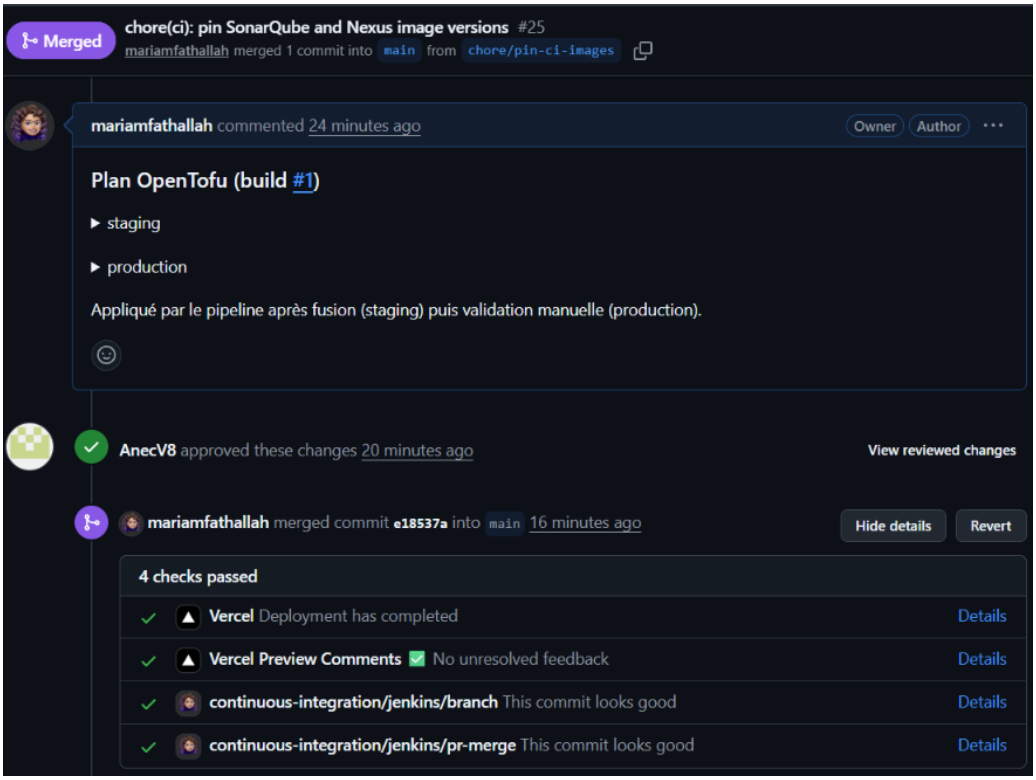


FIGURE 3.2 – Plan OpenTofu publié sur la PR, approbation et checks Jenkins verts

Quand plusieurs commits arrivent sur la même branche, seul le dernier est construit (`disableConcurrentBuilds(abortPrevious: true)`).

L'application étant déjà en ligne avant le projet (Render et Vercel déployaient `main`), le site public a été déplacé sur une branche `live`. `main` est ainsi devenue la branche du projet : chaque fusion y est publiée, déployée en staging, puis en production après validation. Cette migration a été fusionnée par *merge commit*, seule exception à la règle du squash, pour que les tags existants restent atteignables depuis `main` et que semantic-release poursuive la numérotation.

3.3 Shared Library

La bibliothèque (`https://github.com/l3miage-morabitb/jenkins-shared-library`) expose trois fonctions dans son dossier `vars/`. Le nom du fichier est le nom de la fonction.

Fonction	Rôle	Version
`notifyDiscord()`	message de fin de build (statut, lien), dans `post { always }`	v1.0.1
`publishToNexus()`	envoi du jar versionné dans Nexus	v1.1.0
`deployTo(env)`	déploiement via `ci/scripts/deploy.sh`, qui reste inchangé	v1.2.0

TABLE 3.2 – Fonctions de la Shared Library

Versionnage. Ajouter une fonction fait monter le deuxième chiffre, et un tag publié n'est jamais déplacé. La version v1.0.0 contenait une fonction `notify`, renommée `notifyDiscord` en v1.0.1 : Jenkins appelait la méthode Java native `Object.notify()`, refusée par le sandbox.

Secrets. La bibliothèque ne contient aucun secret. Elle lit les credentials Jenkins (`nexus-creds`, `discord-webhook`), créés par JCasC à partir de `ci/.env`.

3.4 Gestion des secrets

Aucun secret n'est écrit dans le dépôt. Chaque membre les place dans un fichier `ci/.env`, exclu par le `.gitignore` ; seul le modèle `ci/.env.example`, sans valeurs, est versionné. Au démarrage, JCasC transforme ces variables en **credentials Jenkins** :

Credential	Type	Utilisation
`sonar-token`	texte secret	analyse SonarQube (`withSonarQubeEnv`)
`github-token`	utilisateur + token	lecture du dépôt, statut des PR, commentaire du plan, releases
`nexus-creds`	utilisateur + mot de passe	publication et téléchargement des artefacts
`discord-webhook`	texte secret	notifications de fin de build

TABLE 3.3 – Credentials Jenkins créés par JCasC

Le pipeline n'y accède que par `credentials()` et `withCredentials`, et Jenkins masque leur valeur dans les journaux (`****`). Le token GitHub est un token *fine-grained*, limité au seul dépôt : en lecture seule pour les membres, et avec les droits d'écriture nécessaires (statuts, commentaires, releases) pour le seul Jenkins de référence.

Les secrets des environnements (mot de passe de la base, clé JWT) suivent un autre chemin : ils sont générés par OpenTofu, conservés dans son état (volume `tf_state`, hors Git), puis transmis à Ansible par un fichier temporaire supprimé après le déploiement, avec des tâches en `no_log`.

Limite. Ces secrets d'environnement ne sont pas des credentials Jenkins : ils restent dans l'état OpenTofu. Une solution plus robuste serait un coffre-fort de secrets (Vault, par exemple) partagé par OpenTofu et Jenkins.

4 Environnements et déploiement

4.1 Staging et production

Staging et production sont décrits par **le même code** (`infra/terraform`) ; seules les valeurs changent, dans `environments/staging.tfvars` et `environments/production.tfvars` (nom et port). Chaque environnement est isolé :

Ressource	Rôle
`tracker-<env>-net`	réseau Docker propre à l'environnement
`tracker-<env>-db-data`	volume des données PostgreSQL, protégé contre la destruction (`prevent_destroy`)
`tracker-<env>-db`	PostgreSQL 16 avec vérification de santé (`pg_isready`)
Mot de passe de la base, clé JWT	générés par OpenTofu (32 et 64 caractères ; la clé JWT doit dépasser 256 bits)
`tracker-<env>-proxy`, `-blue`, `-green`	proxy et application, créés par Ansible (section 4.3) ; seul le proxy est publié, sur le port 8082 (staging) ou 8083 (production)

TABLE 4.1 – Ressources d'un environnement

Chaque ressource porte les labels `project`, `environment` et `managed-by` (`opentofu` ou `ansible`), ce qui permet de voir avec `docker ps` quel outil gère quoi.

4.2 Environnements as Code (OpenTofu et Ansible)

Répartition. OpenTofu crée l'infrastructure durable de chaque environnement (section précédente). Ansible y installe l'application : téléchargement du jar depuis Nexus, construction de l'image, conteneurs blue/green, proxy et vérifications de santé.

Dans le pipeline.

— *Infra plan*, sur toutes les branches : `infra.sh plan` calcule le plan de staging et de production, l'enregistre (`<env>.tfplan`) et l'archive en texte (`plan-<env>.txt`). Sur une pull request, il est publié en commentaire : le relecteur voit l'effet d'un changement d'infrastructure avant la fusion.

— *Deploy staging*, sur `main` : `infra.sh apply staging` applique exactement le plan enregistré plus tôt dans le même build, puis Ansible déploie la version.

— *Deploy production* : même chose, après la validation manuelle.

Passage des informations. `deploy.sh` lit les outputs d'OpenTofu (réseau, nom de la base, secrets), les écrit dans un fichier temporaire lisible par son seul propriétaire, supprimé à la fin, puis lance le playbook. Les tâches qui manipulent des secrets sont en `no_log`. Le fichier `.terraform.lock.hcl` est versionné : tout le monde utilise exactement les mêmes versions de providers.

Vérifications. Relancer un plan sans changement de code affiche « No changes ». Supprimer du code le volume de données fait échouer le plan, grâce à `prevent_destroy`.

4.3 Déploiement blue/green

Le déploiement est réalisé par le playbook `infra/ansible/bluegreen.yml` :

1. lecture de la couleur active, mémorisée dans le proxy ; la nouvelle version est déployée sur la couleur **inactive**, sans toucher au trafic ;
2. vérification de santé (`/actuator/health`) directement sur cette couleur, depuis le réseau de l'environnement ;
3. si elle est correcte : bascule du proxy (`nginx -t` puis `nginx -s reload`, sans interruption), puis nouvelle vérification à travers le proxy ;
4. si elle échoue : la couleur défaillante est supprimée et le proxy reste (ou revient) sur l'ancienne. C'est le **retour arrière automatique** : les utilisateurs ne voient jamais la version défaillante, et le build échoue ;
5. l'ancienne couleur reste démarrée, pour permettre un retour arrière instantané.

Test	**Résultat**
Premier déploiement	`blue` démarré, proxy créé
Nouvelle version	déployée sur `green`, bascule du proxy, `blue` conservée
Même version redéployée	aucune modification (idempotence)
Échec volontaire (base de données arrêtée)	santé en échec, couleur défaillante supprimée, le proxy reste sur l'ancienne couleur (`rescued=1`)
Production depuis `main`	validation manuelle par `admin`, bascule de couleur, `/actuator/health` à `UP`

TABLE 4.2 – Tests du déploiement blue/green

Limites. L'ancienne couleur reste démarrée, ce qui coûte environ 400 Mo de mémoire par environnement. Surtout, les deux couleurs partagent la même base de données et les migrations Liquibase s'exécutent au démarrage de l'application : une migration non rétrocompatible (renommage ou suppression de colonne) casserait l'ancienne version pendant la bascule. L'évolution naturelle serait d'exécuter les migrations dans une étape dédiée du pipeline et d'appliquer le modèle *expand/contract* : ajout, migration des données, puis suppression, sur trois releases.

4.4 Idempotence et même artefact

Le jar est construit une seule fois, publié dans Nexus sous la version `X.Y.Z-<hash>`, puis téléchargé depuis Nexus pour chaque déploiement : staging et production exécutent exactement le même artefact, celui qui a passé les tests. Chaque version est téléchargée dans son propre dossier, ce qui garantit que l'image contient bien le jar de la version annoncée.

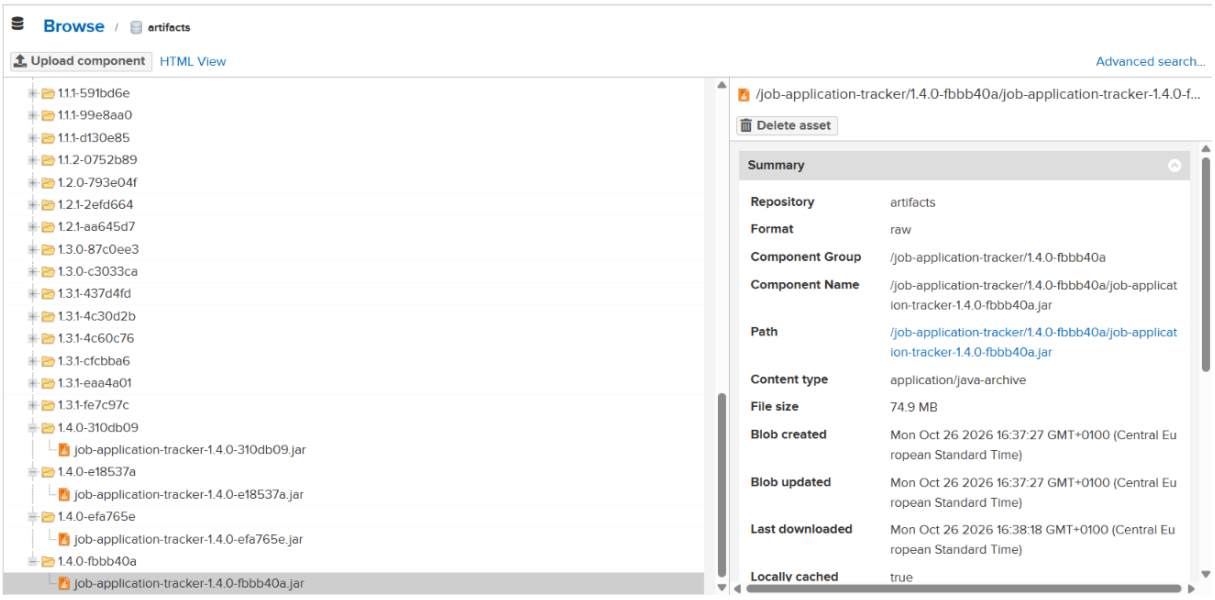


FIGURE 4.1 – Artefacts versionnés `X.Y.Z-<hash>` dans Nexus

Relancer un déploiement ne casse rien : si la version demandée est déjà active, le playbook ne modifie rien ; de même, `tofu plan` sans changement de code affiche « No changes ».

5 Qualité

5.1 Tests et couverture

Le backend est testé avec JUnit lors de `./mvnw verify`, et les résultats sont publiés dans Jenkins. JaCoCo mesure la couverture et génère `jacoco.xml`, lu par Jenkins et par SonarQube. La règle `check` fait échouer le build si la couverture des lignes passe sous le seuil de la propriété `jacoco.minimum` du `pom.xml`, fixé à **50 %**. Ce seuil est un plancher : le contrôle fin est assuré par la Quality Gate (80 % sur le nouveau code).

Sur la branche `main`, la couverture mesurée est de **87,2 %** pour les lignes (259 couvertes, 38 manquées), 83,3 % pour les branches et 88,6 % pour les instructions.

Les tests d'intégration lancent PostgreSQL et l'application avec `docker-compose.test.yml`. Le script `integration-test.sh` attend que `/actuator/health` réponde `UP`, puis vérifie `register` (code 201) et `login` (code 200 avec un jeton). Chaque exécuteur utilise son propre nom de projet Docker et un port choisi par Docker, donc des builds simultanés ne se gênent pas. Les conteneurs sont supprimés dans `post { always }`, même en cas d'échec, et un `.dockerignore` réduit le contexte de construction.

5.2 SonarQube et Quality Gate

L'analyse SonarQube est suivie d'une Quality Gate bloquante (étape `waitForQualityGate` avec `abortPipeline: true`, délai de 5 minutes). La gate utilisée est *Sonar way*, qui suit la méthode *Clean as You Code* et porte sur le nouveau code :

— couverture par les tests d'au moins 80 % ;

— lignes dupliquées : 3 % au plus ;

— notes A en fiabilité, sécurité et maintenabilité ;

— 100 % des *security hotspots* relus.

SonarQube prévient Jenkins par un webhook (`bootstrap-sonar.sh`) ; sans lui, le build attend indéfiniment. Si la gate est rouge, rien n'est publié ni déployé.

5.3 Durée du pipeline

Avec toute la chaîne en place, un build de pull request prenait environ 5 min 30. L'analyse des étapes a montré que SonarQube, l'attente de la Quality Gate et les tests d'intégration s'exécutaient l'un après l'autre (94 à 186 s), alors qu'ils sont indépendants. Ils ont été placés en parallèle, avec `failFast` : le premier échec arrête l'autre branche.

Mesure	Avant	Après
SonarQube + Quality Gate + tests d'intégration	94 à 186 s	environ 65 s
Build complet d'une pull request	environ 5 min 30	environ 3 min
Build de `main` jusqu'au staging	4,2 min	2,7 min

TABLE 5.1 – Durée du pipeline avant et après la parallélisation

Fiabilité. Certains builds échouaient pour des raisons étrangères au code : tests frontend en timeout, agent hors ligne, Docker ne répondant plus. Deux causes ont été mesurées et corrigées :

— **la mémoire** : Nexus occupait environ 2 Go et les conteneurs Java de l'application n'avaient pas de limite. Les plafonds mémoire de la JVM ramènent Nexus à environ 1 Go ;

— **le disque** : l'agent ne dispose que de 3,9 Go, dont 3,1 Go occupés par les workspaces et les jar de toutes les versions. Seul le jar du build est désormais archivé et le workspace est supprimé en fin de build (`deleteDir()`) : il reste 3,5 Go libres.

Limites connues.

— La publication du plan sur la PR nécessite un token GitHub en écriture ; avec un token en lecture seule, l'étape échoue. Le correctif identifié consiste à rendre ce commentaire non bloquant, le plan restant disponible en artefact.

— Dans `infra.sh`, le `tee` masque le code de retour de `tofu plan` : une erreur de plan n'est détectée qu'à l'étape `apply`. Correctif identifié : `set -o pipefail`.

6 Métriques DORA

Les quatre métriques DORA du projet sont calculées automatiquement à partir de l'historique Jenkins et Git, sans aucune saisie manuelle, et suivies dans Grafana sur toute la durée du projet.

6.1 Chaîne de mesure

Prometheus va chercher les métriques auprès de Jenkins (plugin `prometheus`), de l'application en staging et en production (`/actuator/prometheus`) et de l'exporter DORA. Grafana les affiche dans deux tableaux de bord : une vue d'ensemble et le tableau de bord DORA. L'exporter (`ci/dora-exporter`) est la seule pièce écrite pour le module : il lit l'historique Jenkins chaque minute, calcule les quatre métriques et mémorise chaque build dans le volume `dora_data`.

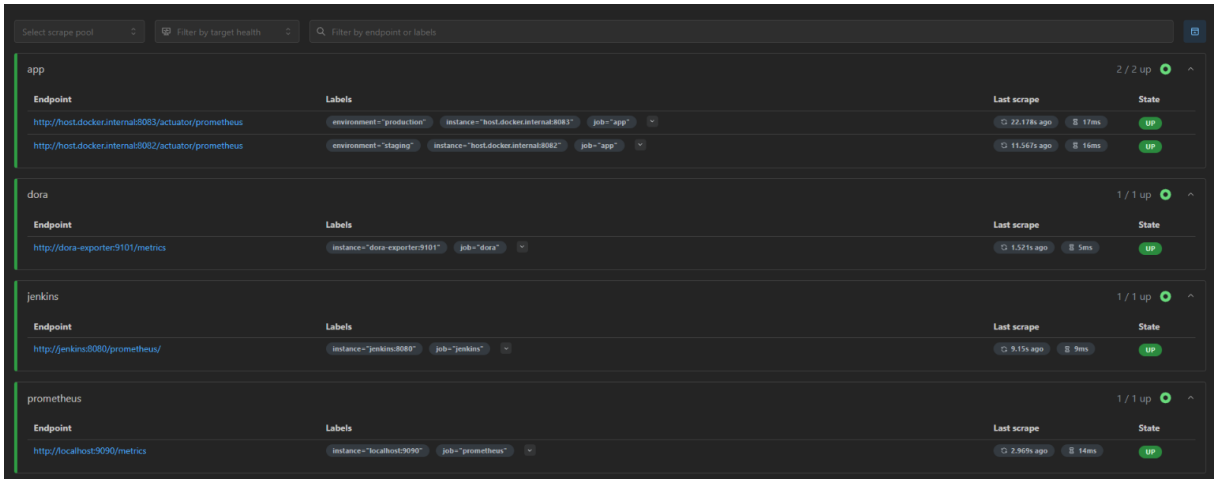


FIGURE 6.1 – Cibles collectées par Prometheus : application (staging et production), exporter DORA, Jenkins

6.2 Définitions et calculs

Un déploiement est l'exécution du stage *Deploy production* d'un build de la branche qui déploie. Il réussit si ce stage finit en succès, et échoue s'il finit en échec. Une validation refusée ou expirée n'est pas un déploiement.

Métrique	**Calcul**	**Niveau élite**
Fréquence de déploiement	déploiements réussis divisés par le nombre de jours de la période	au moins 1 par jour
Délai de livraison	fin du déploiement réussi moins l'heure de chaque commit livré, puis médiane	moins d'un jour
Taux d'échec des changements	déploiements échoués divisés par tous les déploiements	0 à 15 %
Temps de rétablissement	du premier déploiement échoué d'une série au déploiement réussi suivant, puis médiane	moins d'une heure

TABLE 6.1 – Définition des métriques DORA

Trois règles précisent ces calculs :

— **Commits livrés** : un commit construit sans être déployé est rattaché au déploiement réussi suivant. Son délai court donc jusqu'à sa vraie mise en production.

— **Heure d'un commit** : elle vient des *changeSets* de Jenkins. Pour le premier build d'une branche, l'exporter la lit dans l'API GitHub.

— **Retour arrière automatique** : si le blue/green détecte une version défaillante, il rebascule et le stage échoue. Ce cas compte comme un échec.

Chaque métrique est calculée sur trois périodes : 7 jours, 30 jours et toute la durée du projet.

6.3 Résultats

Les valeurs ci-dessous viennent du tableau de bord DORA du Jenkins de référence, relevées le 1er novembre 2026. Elles couvrent environ une semaine de déploiements, d'abord sur la branche `feature/ci-jenkins` puis sur `main`, qui déploie depuis le 26 octobre.

Métrique	Valeur mesurée	Niveau DORA
Fréquence de déploiement	1,99 par jour	élite
Délai de livraison (médiane)	1,5 heure	élite
Taux d'échec des changements	13 % (2 sur 16)	élite
Temps de rétablissement (médiane)	14,7 minutes	élite
Déploiements réussis / échoués	14 / 2	–

TABLE 6.2 – Métriques DORA mesurées

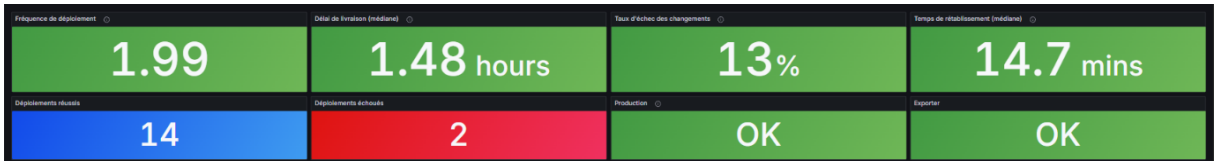


FIGURE 6.2 – Tableau de bord DORA dans Grafana (Jenkins de référence)

6.4 Analyse

Les quatre métriques se situent au niveau « élite » des repères DORA. Ce résultat est à lire avec prudence vu la taille de l'échantillon (voir les limites).

Vitesse. Avec 14 mises en production réussies en une semaine, l'équipe déploie environ deux fois par jour. Chaque pull request fusionnée part en production le jour même. Le délai médian d'une heure et demie entre un commit et sa mise en production est surtout fait d'attente humaine : relecture de la pull request (une approbation obligatoire) et clic de validation avant la production. Le pipeline lui-même dure moins de 10 minutes.

Stabilité. Le tableau de bord compte deux échecs sur 16, chacun rétabli en un quart d'heure environ (14,7 minutes en médiane). En analysant ces deux builds, nous avons constaté qu'aucun ne correspond à une mise en production ratée :

— **après la fusion de la PR #14**, l'agent Jenkins a manqué de mémoire pendant les tests du frontend : il s'est déconnecté et les tests Vitest ont expiré. Le stage *Lint & tests* a échoué et toutes les étapes suivantes ont été sautées. Le build relancé 13,5 minutes plus tard a réussi. Pour éviter que cela se reproduise, la mémoire de Nexus et des conteneurs de l'application a été limitée (PR #17) ;

— **après la fusion de la PR du README**, c'est le déploiement en staging qui a échoué. Les conteneurs de staging et de production étaient arrêtés depuis plusieurs jours : Ansible n'a pas trouvé de couleur active, a recréé le proxy nginx et l'a rechargé avant qu'il ait fini de démarrer (*invalid PID number*). Le build s'est arrêté avant toute bascule et la production a été sautée. Il a été relancé avec succès ; le correctif identifié consiste à réessayer le rechargement de nginx quelques secondes plus tard.

Dans les deux cas, aucune version défaillante n'a été mise en ligne. Mais Jenkins marque comme échoués les stages sautés après une erreur, et l'exporter a donc compté ces deux builds comme des déploiements échoués. Au sens strict de DORA, qui ne retient que les échecs de la mise en production elle-même, le taux d'échec réel de la période est de 0 %, et le temps de rétablissement n'a pas pu être mesuré faute d'incident en production. Les valeurs affichées décrivent plutôt la stabilité de la chaîne de livraison, ce qui reste une information utile : les deux pannes venaient de l'environnement local, pas du code, et ont été résolues en un quart d'heure.

6.5 Limites

— **Petit échantillon** : une douzaine de déploiements sur une semaine. Un seul échec suffit à faire varier fortement le taux d'échec.

— **Période partielle** : Jenkins ne garde que les 20 derniers builds de chaque branche. Quand l'exporter a été mis en service le 26 octobre, les builds plus anciens avaient déjà été supprimés : la mesure ne couvre donc que la dernière semaine du projet.

— **Échec avant la production** : un build qui échoue avant le déploiement est compté comme un déploiement échoué, car Jenkins marque les stages sautés comme échoués. C'est le cas des deux échecs mesurés (voir l'analyse).

— **Une seule instance fait foi** : seul le Jenkins de référence a l'historique complet ; les Jenkins des autres membres ne voient que leurs propres builds.

— **Production locale** : staging et production tournent sur un poste de développement. Un échec peut venir de la machine (mise en veille, mémoire) et non du code.

— **Échec = échec du déploiement** : un bug découvert après une mise en production réussie n'est pas compté.

— **Validation manuelle** : elle allonge le délai de livraison selon la disponibilité de l'équipe.

— **Historique dépendant d'un volume** : supprimer le volume `dora_data` efface les déploiements que Jenkins ne garde plus.

7 Difficultés rencontrées et solutions

Difficulté	Solution
La Quality Gate attendait indéfiniment	ajout du webhook SonarQube vers Jenkins (`bootstrap-sonar.sh`)
Scripts en échec dans les conteneurs à cause des fins de ligne Windows	fins de ligne LF imposées par `.gitattributes`
Docker Desktop bloqué par manque de mémoire	8 Go réservés à WSL, mémoire de Nexus et de l'application limitée
Webhook reçu mais refusé (HTTP 405) : la version 5 de `smee-client` retire le `/` final de l'URL	version épinglée `smee-client@2.0.4`
`ci/.env.example` absent du dépôt, exclu par la règle `.env.*`	exception `!ci/.env.example` dans le `.gitignore`
commitlint 21 et semantic-release 24 exigent deux versions incompatibles d'une même dépendance	commitlint 20, lock régénéré sur l'agent et vérifié par `npm ci`
Notes de version en double après une fusion par *merge commit*	fusion en *squash* uniquement sur `main`
Agent hors ligne, faute de disque	archivage du seul jar du build et `deleteDir()` en fin de build
`main` déployait déjà le site public : impossible d'y déployer la chaîne Jenkins	site public déplacé sur une branche `live`, puis `main` devenue branche de déploiement
Secret Discord créé à la main, absent chez les collègues	déclaré dans `casc.yaml` et `docker-compose.yml`, comme `sonar-token`
Fonction `notify` refusée par le sandbox (`Object.notify`)	renommée `notifyDiscord` (tag v1.0.1)
Tests d'intégration en conflit entre builds parallèles	nom de projet par exécuteur et port choisi par Docker
Contexte de construction Docker trop lourd	`.dockerignore` ciblé
Envoi dans Nexus refusé (403)	Jenkins recréé pour relire `ci/.env`
Les conteneurs lancés par les builds (Docker de l'hôte) ne voyaient pas les fichiers du workspace de l'agent	workspace monté au même chemin (`/opt/jenkins-agent`) côté hôte et côté agent
Clé JWT générée trop courte, refusée par l'application	clé de 64 caractères (plus de 256 bits)
Image construite avec le jar d'une ancienne version (le téléchargement ne remplace pas un fichier existant)	un dossier de téléchargement par version (#16)

Table 7.1 – Difficultés et solutions

8 Répartition du travail

Membre	Réalisations
Basma MORABIT	Shared Library versionnée par tag, tests d'intégration, module Qualité
Bastien LIEVRE	mise en place de l'outillage (Docker Compose, JCasC, agent) et du premier pipeline, module Environnements as Code (OpenTofu, Ansible)
Siham AIDROUS	Module Observabilité et DORA : Prometheus, Grafana, exporter DORA
Mariam FATHALLAH	liaison GitHub, webhook, statut des PR, modules Releases et Blue/green, optimisation et fiabilité du pipeline, migration de la branche de déploiement

TABLE 8.1 – Répartition du travail

Chaque tâche a été réalisée sur sa propre branche (`module/<nom>`, `fix/...`, `docs/...`), puis proposée par pull request vers `main`. Une PR ne peut être fusionnée qu'après un build Jenkins vert et la relecture d'un autre membre ; la fusion se fait en *squash*, avec un message au format Conventional Commits. La documentation d'équipe (`docs/EQUIPE.md`) décrit ce workflow, la répartition des tâches et les problèmes déjà rencontrés, ce qui a permis à chacun de faire tourner la chaîne sur son propre poste.

Conclusion

Bilan

L'objectif du projet est atteint : la partie commune et les cinq modules choisis fonctionnent, dont les deux de niveau 3. Chaque commit sur `main` est construit, testé, analysé, versionné, publié dans Nexus, puis déployé en staging et, après validation, en production, sans aucune action manuelle en dehors de ce clic de validation. Toute la chaîne est décrite dans le dépôt : la plateforme, la configuration de Jenkins, le pipeline et les environnements se recréent à partir du code et d'un fichier `.env`.

Quelques chiffres résument le résultat :

— un build complet d'une pull request dure environ 3 minutes, contre 5 min 30 avant la parallélisation ;

— la couverture des lignes atteint 87,2 %, et la Quality Gate bloque tout code qui ne respecte pas ses conditions ;

— neuf versions ont été calculées et publiées automatiquement par semantic-release ;

— les quatre métriques DORA mesurées se situent au niveau « élite », avec environ deux mises en production par jour.

Ce que le projet nous a appris

Le premier enseignement est qu'une chaîne CI/CD se construit par étapes et se vérifie en continu. Plusieurs problèmes ne sont apparus qu'à l'usage : la mémoire de Docker Desktop, le disque de l'agent, les fins de ligne Windows, ou une version de `smee-client` qui modifiait l'URL du webhook. Chacun a été corrigé dans le code et documenté, pour ne pas se reproduire chez les autres membres.

Le deuxième concerne le travail en équipe. Les règles appliquées au code (une branche par tâche, une pull request relue par un autre membre, un build vert obligatoire) ont évité de casser la branche principale, et la relecture croisée a obligé chacun à comprendre des parties qu'il n'avait pas écrites.

Enfin, mesurer la chaîne nous a appris à interpréter les chiffres avec prudence. L'analyse des métriques DORA a montré que les deux échecs comptés par la mesure n'étaient pas des mises en production ratées, mais des builds arrêtés avant la production. Un indicateur n'a de valeur que si l'on sait exactement ce qu'il compte.

Améliorations possibles

— **Environnements dans le cloud** : remplacer le provider Docker d'OpenTofu par celui d'un hébergeur, la structure du code restant la même, pour ne plus dépendre d'un poste de développement.

— **Migrations de base de données** : les exécuter dans une étape dédiée du pipeline, selon le modèle *expand/contract*, pour rendre le blue/green sûr même en cas de changement de schéma.

— **Secrets** : réunir les secrets de Jenkins et ceux des environnements dans un coffre-fort comme Vault.

— **Mesure DORA** : ne compter comme échec que les erreurs du stage de production, et relier les incidents découverts après une mise en production à la version concernée.

— **Robustesse** : rendre non bloquante la publication du plan sur les pull requests, ajouter `set -o pipefail` au script d'infrastructure et réessayer le rechargement de nginx quand le proxy vient d'être recréé, trois correctifs déjà identifiés.

En quelques semaines, nous sommes passés d'une application déployée à la main à une chaîne qui livre en production plusieurs fois par jour, de façon traçable et mesurée. C'est la base sur laquelle ces améliorations pourraient être construites.